ISTANBUL TECHNICAL UNIVERSITY

COMPUTER ENGINEERING DEPARTMENT

BLG 483E

AI AIDED COMPUTER ENGINEERING HOMEWORK 1 REPORT

CRN : 25311

LECTURER : Dr. Yusuf Hüseyin Şahin

Mustafa Eren KOÇ

150190805

SPRING 2026

Contents

1 INTRODUCTION **1**

2 ASSIGNMENT ANALYSIS **1**

2.1 Problem Restatement 1

2.2 Explicit Technical Constraints 2

2.3 Design Implications 2

3 SYSTEM GOALS **3**

4 METHODOLOGY **3**

4.1 Development Strategy 3

4.2 Reason for Choosing Python Standard Library Components 4

5 SYSTEM ARCHITECTURE **4**

5.1 High-Level Architecture 4

5.2 Why SQLite Was Chosen 5

5.3 Main Architectural Decision: Persisted Frontier + Bounded In-Memory Queue 5

6 DATA MODEL **6**

6.1 Why Two Kinds of Identity Are Stored 6

7 IMPLEMENTATION DETAILS **7**

7.1 URL Normalization 7

7.2 HTML Parsing 7

7.3 Crawl Job Lifecycle 8

7.4 Duplicate Crawl Prevention 8

7.5 Indexing Strategy 9

7.6 Search Strategy 9

7.7 Quiz Compatibility Layer 9

7.8 HTTP API 10

7.9 Dashboard 10

8 CONCURRENCY AND BACK PRESSURE **11**

8.1 Why Concurrency Matters in This Assignment 11

8.2 Thread Model 11

8.3 Back Pressure Mechanisms 12

8.3.1 Rate Limiting 12

8.3.2 Bounded Queue Depth . . . 12
8.4 How Search Runs While Indexing . . . 12

9 PERSISTENCE AND RESUME SUPPORT **12**
9.1 Interruption Handling . . . 13
9.2 Why This Matters . . . 13

10 RESULTS AND VALIDATION **13**
10.1 Automated Validation . . . 13
10.2 Live Localhost Validation . . . 14
10.3 Observed Improvement During Validation . . . 14
10.4 Example Result . . . 14
10.5 Quiz Validation Example . . . 15

11 DISCUSSION **15**
11.1 Strengths of the Final Solution . . . 15
11.2 Tradeoffs and Limitations . . . 16
11.3 Why the Chosen Scope Is Appropriate . . . 16

12 PRODUCTION RECOMMENDATIONS **16**
12.1 Separate Fetch, Index, and Query Tiers . . . 17
12.2 Adopt Stronger Crawl Policy Controls . . . 17
12.3 Move Beyond SQLite for Search Scale . . . 17

13 CONCLUSION **18**

1 INTRODUCTION

This report presents the design, implementation, validation, and discussion of Homework 1 for BLG 483E AI Aided Computer Engineering. The assignment asks for a localhost-runnable web crawler that provides two major capabilities: an `index(origin, k)` operation for crawling the web up to depth `k`, and a `search(query)` operation that returns relevant results while indexing is still ongoing. In addition, the system must expose the state of the crawler in a user-facing UI or CLI, explicitly reflecting progress, queue depth, and some notion of back pressure. Persistence and resume support are optional but considered a strong plus.

The solution developed for this homework is a single-machine crawler and search engine implemented primarily with Python standard-library components. The system deliberately avoids high-level crawler frameworks and instead relies on `urllib.request`, `html.parser`, `threading`, `queue.Queue`, `sqlite3`, and `http.server`. This choice directly aligns with the assignment requirement that language-native tools should be used "to the greatest extent possible" rather than external libraries that solve the core exercise out of the box.

From an architectural perspective, the implemented system aims to balance correctness, clarity, and practical engineering judgment under a constrained project scope. The crawler is built as a persisted frontier plus in-memory worker queue design. Search is executed directly against a transactional SQLite index running in WAL mode, which allows the system to expose newly indexed pages immediately while crawl workers continue to discover more content. As a result, the implementation not only satisfies the functional requirements but also meaningfully addresses the assignment's explicit design prompt: how can search continue to operate while indexing is active?

This report first restates the assignment requirements in engineering terms, then explains the system architecture, implementation details, concurrency and back-pressure strategy, persistence model, validation process, and final outcomes. It concludes with a discussion of design tradeoffs and recommendations for how the current localhost prototype could be evolved into a production-oriented crawler system.

2 ASSIGNMENT ANALYSIS

2.1 Problem Restatement

The assignment defines two externally visible operations:

- **Index**: Given an origin URL and a maximum depth `k`, start a web crawl such that

no page is crawled twice. Depth is defined as the number of hops from the origin to the discovered page.

- **Search**: Given a query string, return relevant results as triples of the form `(relevant_url, origin_url, depth)`.

Although the assignment states that for the exercise we may assume `index` is invoked before `search`, it also explicitly asks for design thoughts on supporting search while indexing is still active. Therefore, a purely offline batch-index-then-search design would be acceptable only in the narrowest interpretation of the first statement, but it would fail to engage with one of the most important architectural questions embedded in the prompt. For that reason, the implementation intentionally supports true incremental indexing and concurrent search visibility.

2.2 Explicit Technical Constraints

Several constraints materially shape the design:

- The crawler should scale to a large crawl on a **single machine**.
- The system must include **back pressure**, such as queue limits or rate limits.
- The system should avoid **duplicate crawling**.
- The implementation should prefer **native language facilities** over turnkey crawler libraries.
- The system should provide a **simple UI or CLI**.
- A resume-after-interruption capability is a **bonus**, not a hard requirement, but valuable.

2.3 Design Implications

These constraints imply several concrete engineering decisions:

- A single-machine architecture should prefer a durable local store rather than distributed infrastructure.
- Search results should be committed incrementally, not only at the end of a crawl job.
- The queue that represents work in flight should be bounded to avoid uncontrolled memory growth.

- URL deduplication should persist beyond process memory if resume behavior is desired.
- The system needs to expose runtime information that is meaningful to a reviewer, not just internal debug logs.

Taken together, the prompt is less about raw crawling volume and more about architectural sensibility: correctness under concurrency, bounded load, readable operational state, and a thoughtful tradeoff between simplicity and extensibility.

3 SYSTEM GOALS

Based on the assignment, the implemented system was designed around the following goals:

1. Correctly crawl reachable HTTP/HTTPS pages up to the requested maximum depth.
2. Ensure a page is fetched at most once by the system unless data is explicitly cleared.
3. Expose partial search results while crawl jobs are still running.
4. Bound work in memory through explicit queue capacity and request pacing.
5. Persist sufficient state so interrupted jobs can be resumed.
6. Offer a simple, clear localhost dashboard for crawl creation, monitoring, and search.
7. Keep the implementation dependency-light and explainable.

4 METHODOLOGY

4.1 Development Strategy

The implementation followed a pragmatic staged approach:

- First, the assignment brief and reference materials were read and translated into technical requirements.
- Second, the system architecture was defined before writing the crawler logic itself.
- Third, core backend capabilities were implemented: storage, URL normalization, parsing, crawling, indexing, search, and runtime job management.

- Fourth, a lightweight dashboard was built to make the system observable and easy to operate locally.
- Fifth, the system was validated with automated integration tests and live localhost runs.

4.2 Reason for Choosing Python Standard Library Components

The final code intentionally uses standard-library primitives because the assignment strongly encourages native functionality. The mapping between requirements and chosen tools is summarized below:

Table 1: Core technologies used in the implementation

Technology	**Role in the System**
`urllib.request`	HTTP fetching and redirect-aware page retrieval
`html.parser`	Extraction of text, title, and links from HTML content
`threading`	Dispatcher thread, worker threads, and coordination primitives
`queue.Queue`	Bounded in-memory work queue for back pressure
`sqlite3`	Persistent frontier, page store, inverted index, events, and job metadata
`http.server`	Local API and dashboard hosting without external web frameworks

This choice keeps the system transparent. Each major behavior is directly visible in the source code and is not delegated to a framework abstraction that would obscure the design tradeoffs.

5 SYSTEM ARCHITECTURE

5.1 High-Level Architecture

The implemented crawler is organized into five principal modules:

1. **Entry point** (`main.py`)
2. **HTTP layer** (`crawler_app/http_server.py`)
3. **Crawl manager and job runner** (`crawler_app/manager.py`)

4. **HTML parsing and utility logic** (`crawler_app/parser.py`, `crawler_app/utils.py`)
5. **Persistent storage layer** (`crawler_app/storage.py`)

At runtime, the overall control flow is:

1. A user sends `POST /api/index` with `origin`, `max_depth`, and capacity parameters.
2. A job record is created in SQLite, and the frontier is seeded with the origin URL at depth zero.
3. A dispatcher thread moves pending frontier items into a bounded in-memory queue.
4. Worker threads dequeue URLs, fetch pages, parse links and text, store the page in SQLite, and enqueue newly discovered links if depth allows.
5. Search requests query the already committed index directly from SQLite, even if more pages are still being processed.

5.2 Why SQLite Was Chosen

SQLite is a strong fit for this scope because:

- it is local and requires no additional service installation,
- it provides transactional guarantees,
- WAL mode supports concurrent readers during writes,
- it keeps the persistence story simple and explicit,
- it is sufficient for a single-machine prototype with moderate write concurrency.

This design is particularly useful for the "search while indexing" requirement. Instead of waiting for a crawl to finish and then bulk-loading data into a separate index, every processed page becomes searchable as soon as its transaction commits.

5.3 Main Architectural Decision: Persisted Frontier + Bounded In-Memory Queue

One of the most important design choices in the system is the split between:

- a **persistent frontier** stored in SQLite, and
- a **bounded in-memory work queue** used only for active work.

This division solves two problems at once:

1. It prevents memory growth from exploding when a page discovers many outgoing links.
2. It makes interruption and resume straightforward because undispatched work remains durable.

This is a stronger single-machine design than simply storing everything in an in-memory queue and writing results opportunistically to disk.

6 DATA MODEL

The SQLite schema is central to the implementation. The key tables are summarized below.

Table 2: Persistent tables used by the crawler

Table	**Purpose**
`jobs`	Stores metadata about crawl jobs, including depth, queue limit, status, timestamps, and error state
`frontier`	Stores discovered URLs per job, their depth, parent URL, and lifecycle state (`pending`, `queued`, `in_progress`, `done`, `error`, `skipped`)
`pages`	Stores globally fetched pages, response metadata, title, plain text, and fetch error state
`url_aliases`	Maps redirected request URLs onto their canonical stored page URLs
`page_terms`	Stores per-page term frequency and whether a term appears in the title
`page_links`	Stores outgoing links extracted from a page
`page_origins`	Stores how a page relates to a specific crawl job, including origin URL and discovered depth
`job_events`	Stores timestamped operational messages for dashboard visibility

6.1 Why Two Kinds of Identity Are Stored

The assignment requires that search return not only a relevant page URL, but also the origin and depth under which the page was discovered. For that reason, the data model distinguishes:

- **global page identity**: the canonical URL stored in `pages`, and
- **job-specific discovery identity**: the `origin_url` and `discovered_depth` stored in `page_origins`.

This is a crucial modeling decision because the same physical page may be reachable from multiple crawl jobs, multiple origins, or different depths. A simple page-only table would not be enough to return the required result triples.

7 IMPLEMENTATION DETAILS

7.1 URL Normalization

All candidate URLs are normalized before insertion into the frontier. The normalization procedure:

- resolves relative links using the current page URL as a base,
- keeps only `http` and `https` schemes,
- lowercases the scheme and hostname,
- removes fragment identifiers,
- normalizes default ports,
- canonicalizes the path.

This reduces duplicate frontier entries caused by syntactic differences that refer to the same resource.

7.2 HTML Parsing

The parser is intentionally minimal and native. It subclasses `HTMLParser` and:

- ignores `script`, `style`, and `noscript` content,
- extracts title text separately,
- collects body text,
- collects `href` attributes from anchor tags,
- normalizes discovered links to absolute URLs.

The parser returns a tuple of:

- page title,
- plain text content,
- deduplicated outgoing links.

7.3 Crawl Job Lifecycle

Each crawl job progresses through the following stages:

1. Job creation and frontier seeding.
2. Dispatcher startup.
3. Worker thread startup.
4. Repeated claim-fetch-parse-store-expand cycle.
5. Job completion when no pending, queued, or in-progress frontier entries remain.

The dispatcher and workers have intentionally separate responsibilities:

- the dispatcher is responsible for moving work from disk into bounded memory,
- workers are responsible for executing that work.

This separation keeps control flow simpler and makes queue pressure visible and measurable.

7.4 Duplicate Crawl Prevention

Preventing duplicate crawling is a core assignment requirement. In this implementation, duplicate prevention happens at two levels:

1. **Persistent deduplication**: the `pages` table stores each canonical page URL only once.
2. **Concurrent fetch coordination**: if multiple threads encounter the same not-yet-fetched URL at nearly the same time, an in-memory coordination map ensures that only one thread performs the actual network fetch while others wait for the result.

This design is stronger than a per-job visited set, because it avoids re-fetching a page even across multiple crawl jobs in the same persisted database.

7.5 Indexing Strategy

After a successful HTML fetch:

1. the text is tokenized to lowercase alphanumeric terms,
2. per-term frequency is computed,
3. title terms are marked separately,
4. the page and its term frequencies are committed to SQLite,
5. outgoing links are stored for later depth expansion.

Each page is committed independently. This is the key enabler for live search visibility.

7.6 Search Strategy

Search uses a simple and explainable relevance model. Query text is tokenized the same way indexed page text is tokenized. A SQL query then:

- looks up matching terms in `page_terms`,
- joins them with `pages` and `page_origins`,
- groups results by `(relevant_url, origin_url, depth)`,
- computes a score using term frequency and title matches,
- orders results by number of matched query terms, score, shallower depth, and URL.

The scoring formula is intentionally simple:

```
score = 4 * frequency + 20 * title_match_bonus
```

This model is not intended to compete with industrial ranking pipelines, but it is deterministic, easy to explain, and sufficient for the assignment.

7.7 Quiz Compatibility Layer

After the main crawler was completed, the project was additionally aligned with a course quiz that expected two concrete interfaces:

- a raw storage file at `data/storage/p.data`,

- a search endpoint exposed as `GET /search?query=... & sortBy=relevance` on port `3600`.

The core system still stores its authoritative index in SQLite. Therefore, instead of replacing the primary architecture, a compatibility export was added that materializes flat-file rows directly from the canonical index in the following format:

```
word url origin depth frequency
```

In parallel, a quiz-specific search route was introduced so that the returned score matches the quiz formula exactly:

```
score = (frequency * 10) + 1000 - (depth * 5)
```

This compatibility layer is intentionally separate from the richer dashboard search API. As a result, the original homework design remains intact while the repository also satisfies the quiz workflow exactly.

7.8 HTTP API

The localhost dashboard communicates with the crawler through a small JSON API:

Table 3: Implemented HTTP endpoints

Endpoint	**Method**	**Purpose**
`/api/index`	POST	Create and start a new crawl job
`/api/status`	GET	Return system-level summary information
`/api/jobs`	GET	List all jobs and their statuses
`/api/jobs/{job_id}`	GET	Return detailed state for a specific job
`/api/jobs/{job_id}/resume`	POST	Resume a resumable job
`/api/search?q=...`	GET	Search indexed content and return result triples
`/search?query=... & sortBy=relevance`	GET	Quiz-compatible search using the exported flat-file scoring formula

7.9 Dashboard

The dashboard was implemented as a lightweight single-page front end served by the same local HTTP server. It supports:

- creating crawl jobs,

- listing jobs,
- selecting a job to inspect detailed state,
- running search queries,
- seeing operational signals such as indexed page count, active workers, queue depth, and back-pressure events.

The final UI also includes a summary section explaining:

- what the homework expects,
- what has been implemented,
- how the dashboard should be used during review.

8 CONCURRENCY AND BACK PRESSURE

8.1 Why Concurrency Matters in This Assignment

Concurrency appears in two places:

- multiple URLs should be processed in parallel to use a single machine effectively,
- search and indexing should be able to coexist safely.

However, uncontrolled concurrency would create correctness and resource problems. The solution therefore uses carefully bounded concurrency instead of unconstrained thread creation.

8.2 Thread Model

Each active crawl job consists of:

- one dispatcher thread,
- a configurable number of worker threads,
- the main HTTP server thread(s) serving dashboard and search traffic.

The storage layer protects write operations with a reentrant lock and uses SQLite WAL mode to keep reads available during writes.

8.3 Back Pressure Mechanisms

Back pressure is implemented in two complementary ways:

8.3.1 Rate Limiting

A job-specific rate limiter enforces a maximum request rate. This directly addresses the assignment's suggestion that back pressure can be implemented as a maximum rate of work.

8.3.2 Bounded Queue Depth

The in-memory queue has a fixed maximum size. If the queue is full, the dispatcher does not continue loading more work from the persistent frontier. This means:

- memory consumption stays bounded,
- discovered work can remain on disk safely,
- the dashboard can show a meaningful queue-pressure signal.

8.4 How Search Runs While Indexing

The assignment explicitly asks for design thinking on supporting search while indexing is still active. In this implementation, that capability is achieved through:

1. page-by-page transactional commits,
2. a durable SQL-backed inverted index,
3. SQLite WAL mode, which allows concurrent reads while writes are happening.

This is substantially better than a batch-indexing design because the search endpoint does not need to wait for crawl completion. As soon as a page commit finishes, it becomes searchable.

9 PERSISTENCE AND RESUME SUPPORT

Resume behavior is implemented as a first-class capability even though it was optional in the assignment.

9.1 Interruption Handling

If the process stops while jobs are still active, unfinished work is not discarded. On startup:

- jobs that were previously `running` are marked `resumable`,
- frontier items that were `queued` or `in_progress` are reset to `pending`,
- if `-auto-resume` is enabled, resumable jobs are restarted automatically.

9.2 Why This Matters

Without persisted frontier state, interruption would require restarting the crawl from the origin. That would be wasteful, especially for deeper crawls. Resume support is therefore not just a bonus convenience; it is a meaningful improvement to the system's engineering quality.

10 RESULTS AND VALIDATION

10.1 Automated Validation

The project includes integration tests that validate three important behaviors:

1. crawl completion and indexed search results,
2. live search visibility while a crawl is still running,
3. resuming interrupted jobs,
4. quiz-compatible raw export and exact relevance-score calculation.

The executed test command was:

```
python -m unittest discover -s tests -v
```

The test suite passed successfully. This is important because it verifies not only nominal crawling behavior but also the core architectural requirements around concurrency and resumability.

10.2 Live Localhost Validation

In addition to automated tests, the system was run interactively on localhost. The following setup was used:

```
python -m http.server 9001 -d sample_site
python main.py --host 127.0.0.1 --port 3600 --auto-resume
```

During live validation:

- the sample site was served at http://127.0.0.1:9001/index.html,
- the dashboard and API were served at http://127.0.0.1:3600,
- a crawl job with maximum depth 2 completed successfully,
- three pages were indexed,
- the query `python` correctly returned `page-a.html` with origin `index.html` and depth 1,
- the compatibility export file `data/storage/p.data` was generated successfully,
- the compatibility query `/search?query=relevance&sortBy=relevance` returned `page-b.html` with `relevance_score = 1025`.

10.3 Observed Improvement During Validation

During live testing, a duplicate result issue was observed in search responses when the same page had been associated with multiple crawl jobs that shared the same origin and depth. The issue was traced to grouping behavior in the search SQL query. The query was then corrected to group results by `(relevant_url, origin_url, depth)` rather than by crawl job identity. This debugging step improved both correctness and presentation quality.

10.4 Example Result

A representative search response after the fix was:

```
query: python
result_count: 1
relevant_url: http://127.0.0.1:9001/page-a.html
origin_url:   http://127.0.0.1:9001/index.html
depth:        1
score:        64
```

This output is consistent with the assignment requirement that search returns the relevant page together with the origin and discovery depth that explain how the page entered the index.

10.5 Quiz Validation Example

The quiz compatibility path was also verified directly against the exported flat file. The chosen word was `relevance`, which appeared on three distinct URLs:

```
relevance http://127.0.0.1:9001/index.html  http://127.0.0.1:9001/index.html 0 1
relevance http://127.0.0.1:9001/page-a.html http://127.0.0.1:9001/index.html 1 1
relevance http://127.0.0.1:9001/page-b.html http://127.0.0.1:9001/index.html 1 3
```

Applying the quiz formula gives:

- `index.html`: $(1 \times 10) + 1000 - (0 \times 5) = 1010$
- `page-a.html`: $(1 \times 10) + 1000 - (1 \times 5) = 1005$
- `page-b.html`: $(3 \times 10) + 1000 - (1 \times 5) = 1025$

The top API result from `GET /search?query=relevance&sortBy=relevance` was `page-b.html` with `relevance_score = 1025`, which matches the manual calculation exactly.

11 DISCUSSION

11.1 Strengths of the Final Solution

The strongest aspects of the implementation are the following:

- **Requirement alignment**: the solution directly addresses the assignment's core demands rather than building unrelated extra features.
- **Native implementation**: the system stays faithful to the prompt's preference for language-native functionality.
- **Operational clarity**: queue pressure, worker activity, job events, and search results are visible through a straightforward dashboard.
- **Correctness under concurrency**: duplicate fetch coordination and WAL-backed incremental indexing make the system robust enough for the homework scope.
- **Persistence**: the resume behavior significantly improves practical usability.

11.2 Tradeoffs and Limitations

The current solution is intentionally scoped. Several limitations are acceptable for a homework deliverable but would matter in a larger system:

- **No robots.txt handling**: the crawler does not yet enforce web politeness policies.
- **Simple ranking**: the relevance model is intentionally basic.
- **Single-machine ceiling**: SQLite is appropriate here, but not the right long-term choice for very large, write-heavy deployments.
- **HTML-only focus**: non-HTML content is detected but not richly processed.
- **No recrawl scheduling**: a URL is not revisited unless data is reset.

11.3 Why the Chosen Scope Is Appropriate

The assignment explicitly emphasizes thoughtful code that works within reasonable assumptions over an expansive but incomplete system. The final implementation reflects that advice. Instead of adding distributed components, advanced ranking heuristics, or an overbuilt microservice architecture, the system focuses on:

- correctness,
- bounded resource usage,
- concurrency-aware indexing,
- observability,
- and local reproducibility.

That tradeoff is, in my view, the right one for a 3–5 hour assignment with architectural evaluation criteria.

12 PRODUCTION RECOMMENDATIONS

Although the project is intentionally single-machine and localhost-oriented, several next steps would be appropriate for productionization.

12.1 Separate Fetch, Index, and Query Tiers

The first major recommendation is to split the current unified process into separate fetch, index, and query responsibilities:

- fetch workers retrieve and normalize raw page documents,
- index workers transform documents into search-ready postings,
- query nodes serve read traffic against replicated search infrastructure.

This would reduce write-read interference, improve fault isolation, and let each subsystem scale according to its own bottleneck.

12.2 Adopt Stronger Crawl Policy Controls

A production crawler should support:

- `robots.txt` awareness,
- per-host politeness budgets,
- retry budgets and exponential backoff,
- page freshness and recrawl policies,
- abuse detection and monitoring.

12.3 Move Beyond SQLite for Search Scale

SQLite is excellent for this project, but a production environment should move toward:

- a durable queue or log for fetched documents,
- a distributed index store such as OpenSearch or Elasticsearch,
- or a dedicated custom inverted-index service.

This would make it possible to add ranking signals such as phrase matching, recency, anchor text, and authority-based scoring.

13 CONCLUSION

The objective of this homework was to build a web crawler and search engine that satisfies both algorithmic and architectural requirements: crawl from an origin URL up to depth `k`, avoid crawling the same page twice, apply back pressure, provide live search over indexed content, and expose operational state through a UI or CLI. The final implementation achieves these goals with a clear, dependency-light architecture built almost entirely on Python standard-library components.

More importantly, the project addresses the assignment's deeper systems question about concurrent indexing and search in a concrete and defensible way. Instead of deferring search until the crawl is finished, the solution writes pages transactionally into a SQLite-backed inverted index and serves search directly from that live store. Together with the persisted frontier, bounded in-memory queue, duplicate-fetch coordination, and resume support, the resulting system is both practically useful and architecturally sensible for a single-machine homework deliverable.

In summary, the project satisfies the required functionality, demonstrates careful attention to detail, and remains faithful to the design constraints stated in the prompt. It is therefore a complete and technically coherent submission for the assignment.

REFERENCES

[1] Python Software Foundation. *urllib.request — Extensible library for opening URLs.* https://docs.python.org/3/library/urllib.request.html

[2] Python Software Foundation. *html.parser — Simple HTML and XHTML parser.* https://docs.python.org/3/library/html.parser.html

[3] Python Software Foundation. *sqlite3 — DB-API 2.0 interface for SQLite databases.* https://docs.python.org/3/library/sqlite3.html

[4] Python Software Foundation. *http.server — HTTP servers.* https://docs.python.org/3/library/http.server.html

[5] SQLite Documentation. *Write-Ahead Logging.* https://www.sqlite.org/wal.html

[6] Istanbul Technical University, BLG 483E Homework 1 brief and supporting course materials provided with the project package.